Промежуточная аттестация. Раздел 1. Приложение «Каталог»

Финальная сборка вашего каталога. Всё, что вы делали в ДЗ 2–6, собирается в одно законченное приложение: вход по аккаунту, каталог в Firebase, экран деталей, управление состоянием через BLoC.

Обязательная часть почти целиком состоит из того, что уже есть в ваших домашках. Новое здесь: экран деталей с навигацией, аккуратные состояния экрана и одна дополнительная опция на ваш выбор. Дальше можно добавлять что угодно от себя: креатив только приветствуется.

Дедлайн: воскресенье, 1 ноября, 23:59 по МСК.

1. Что нужно сделать

Основа — ваш каталог из домашек: та же тема, та же модель. Код живёт в `lib/`, как и раньше.

Часть 1. Вход в приложение (Firebase Authentication)

1. Экран входа и регистрации по email и паролю.
2. Без входа каталог недоступен: после входа пользователь попадает в каталог.
3. Есть кнопка «Выйти», после выхода — снова экран входа.
4. После перезапуска приложения пользователь остаётся залогиненным, вводить пароль заново не нужно.
5. Ошибки входа и регистрации (неверный пароль, email уже занят, слишком короткий пароль) показываются на экране понятным текстом, а не только в консоли.

Часть 2. Экран каталога

1. Прокручиваемый список из 8 и более элементов. В карточке: обложка, название, подпись, иконка категории и кнопка лайка.
2. Лайк переключается, фильтр «Только избранные» работает.
3. У экрана три состояния. **Загрузка** — индикатор. **Ошибка** — понятный текст и кнопка «Обновить», приложение не падает. **Пустой список** —

заглушка с текстом (например, «В избранном пока пусто»), если элементов нет или фильтр ничего не нашёл.

Часть 3. Детали и добавление (навигация)

1. Тап по карточке открывает экран деталей с полной информацией об элементе: обложка, название, подпись, категория, описание и другие поля вашей модели.
2. С экрана деталей можно вернуться назад. Лайк на экране деталей и в списке совпадает.
3. Добавление элемента — на отдельном экране, в диалоге или в bottom sheet (на выбор). Обязательное поле одно — «Название», пустое название не отправляется.
4. После добавления новый элемент появляется в списке.

Часть 4. Данные (Cloud Firestore)

1. Каталог хранится в коллекции Firestore, а не в коде и не в MockAPI.
2. Добавление элемента создаёт документ в Firestore. После перезапуска приложения элемент на месте.
3. Лайки привязаны к пользователю: после выхода и повторного входа избранное сохраняется, а у другого аккаунта избранное своё.

Часть 5. Архитектура (BLoC)

1. Данными каталога управляет Bloc или Cubit. В виджетах нет `setState` для данных каталога (для локальных мелочей вроде анимаций и полей ввода `setState` можно).
2. У Bloc есть явные состояния: загрузка, данные, ошибка. Есть события или методы: загрузить каталог, лайкнуть, переключить фильтр, добавить элемент.
3. Bloc создаётся через `BlocProvider`, экран строится через `BlocBuilder` (и при необходимости `BlocListener`).
4. Работа с Firebase вынесена в репозиторий: Bloc обращается к репозиторию, виджеты к Firebase напрямую не обращаются.
5. Модель элемента — отдельный класс с преобразованием из документа Firestore и обратно (`fromJson` / `toJson` или аналог). Классы разнесены по файлам.

Часть 6. Качество

1. Приложение собирается и запускается командой `flutter run`.
2. Команда `flutter analyze` не выдаёт ошибок (errors). Предупреждения желательно тоже убрать.

2. Дополнительные опции

Помимо обязательной части **выберите и реализуйте минимум одну опцию** из списка. Можно больше — это плюс к работе. В видео обязательно назовите, какие опции вы сделали, и покажите их.

1. **Фильтры и сортировка.** Фильтр по категории (чипсы или выпадающий список) и сортировка (по названию, по дате добавления). Фильтры сочетаются между собой и с «Только избранными», их состояние хранится в Bloc.
2. **Поиск.** Поле поиска над списком: список фильтруется по названию прямо во время ввода, без учёта регистра. Если ничего не найдено — заглушка.
3. **Редактирование и удаление.** На экране деталей можно изменить элемент и удалить его. Перед удалением — диалог подтверждения. Изменения сохраняются в Firestore.
4. **Профиль.** Отдельный экран профиля: email пользователя, количество элементов в избранном, кнопка выхода и сброс пароля через письмо на почту.
5. **Тёмная тема.** Переключатель светлой и тёмной темы внутри приложения, выбор сохраняется после перезапуска. Все экраны читаются в обеих темах.
6. **Обновление в реальном времени.** Каталог подписан на изменения Firestore: если изменить или добавить документ в Firebase Console, приложение показывает это сразу, без перезагрузки.
7. **Анимации.** Hero-анимация обложки при переходе в детали и анимация нажатия на лайк.
8. **Своя идея.** Любая другая фича сопоставимого объёма. Её нужно заранее согласовать со мной — напишите до 25 октября.

3. Дизайн

Эталонный макет всех экранов — в Figma: https://www.figma.com/design/HV4r52vVUitWuQ1rPHNbyo?node-id=393-2 (страница «Аттестация P1»). В нём показано, какие экраны и элементы должны быть: вход, каталог в разных состояниях, детали, три варианта добавления (bottom sheet, диалог, отдельный экран — выберите любой).

- Макет — ориентир, а не требование к пикселям. Цвета, шрифты, расположение и стиль можно менять как угодно.
- Обязательно должны остаться элементы из раздела 1: например, в карточке — обложка, название, подпись, иконка категории и лайк.
- Добавлять свои экраны, элементы и идеи можно и нужно.

4. Подсказки

- Firebase используем на бесплатном тарифе Spark, привязывать карту не нужно.
- **Не используйте Firebase Storage** для картинок: в новых проектах он требует платный тариф. Обложки делайте как раньше — цветной контейнер с буквой, картинки из `assets` (в Firestore храните путь строкой) или ссылка на картинку в интернете (в Firestore — строка с адресом, в коде `Image.network`).
- Лайки пользователя удобно хранить в Firestore отдельно от каталога, например в документе `users/<uid>` полем-списком id избранных элементов. `uid` текущего пользователя берётся из `FirebaseAuth.instance.currentUser`.
- Проверить, залогинен ли пользователь при запуске, можно через `FirebaseAuth.instance.authStateChanges()` — поток, который сообщает о каждом входе и выходе.
- В правилах Firestore (Rules) разрешите чтение и запись только вошедшим пользователям:

```
allow read, write: if request.auth != null;
```

- Для проверки состояния ошибки отключите интернет на устройстве или эмуляторе и перезапустите загрузку.

5. Видео

Видео **не длиннее 5 минут, с голосом**. Состоит из двух частей примерно поровну.

Часть 1. Демо приложения (2–2,5 минуты)

Покажите по порядку:

1. Регистрацию или вход.
2. Загрузку каталога и скролл списка.
3. Лайк и фильтр «Только избранные», включая пустое состояние.
4. Добавление нового элемента.
5. Открытие экрана деталей и возврат назад.
6. Состояние ошибки (без интернета) и кнопку «Обновить».
7. Выход, повторный вход — избранное на месте.
8. Выбранные дополнительные опции — назовите их вслух.

Часть 2. Как устроен код (2–2,5 минуты)

Откройте проект и коротко пройдитесь по коду: покажите, откуда что берётся и куда идёт. Какие данные есть в приложении и где они хранятся, как они попадают из Firebase на экран и что происходит, когда пользователь что-то нажимает. Не нужно зачитывать каждый файл — важно показать, что вы понимаете, как устроено ваше приложение.

Чтобы уложиться в 5 минут, прорепетируйте демо один раз перед записью и заранее откройте нужные файлы.

6. Как сдать работу через GitHub

Работаем в том же репозитории и проекте, каталог по-прежнему живёт в `lib/`.

Шаг 1. Смерджите последнее ДЗ

Откройте Pull Request по последнему сданному ДЗ на GitHub, нажмите зелёную кнопку Merge pull request, затем Confirm merge. Так в новой ветке будет весь ваш код из домашек.

Шаг 2. Обновите main

Откройте терминал в корне проекта (там, где лежит `pubspec.yaml`) и выполните:

```
git checkout main
git pull origin main
```

Шаг 3. Создайте ветку project

```
git checkout -b project
```

Команда создаёт ветку project и сразу переключает вас на неё.

Шаг 4. Реализуйте задание

Доделайте обязательную часть и выбранные опции. Перед сдачей проверьте:

```
flutter analyze
flutter run
```

Шаг 5. Закоммитьте и отправьте на GitHub

```
git add .
git commit -m "project: final catalog app"
git push -u origin project
```

Коммитить можно и по ходу работы, сколько угодно раз — это даже лучше.

Шаг 6. Откройте Pull Request в main

Зайдите на страницу репозитория на GitHub и нажмите Compare & pull request на жёлтой плашке. Если плашки нет, откройте вкладку Pull requests → New pull request, слева укажите base: main, справа compare: project.

Назовите Pull Request «project» и нажмите Create pull request. Сами этот Pull Request не мерджите: он должен остаться открытым.

Шаг 7. Запишите видео

По плану из раздела 5: демо и рассказ о коде, с голосом, не длиннее 5 минут.

Шаг 8. Сдайте работу в LMS

До дедлайна загрузите на платформу:

- видео (или ссылку на видео, если файл слишком большой);
- ссылку на открытый Pull Request, она выглядит как https://github.com/ваш-логин/название/pull/7;
- список выбранных дополнительных опций одной строкой.

7. Как оценивается

Аттестация — зачёт. Чтобы получить зачёт, нужно:

- выполнить всю обязательную часть (раздел 1);
- реализовать минимум одну дополнительную опцию (раздел 2);
- сдать видео с демо и рассказом о коде (раздел 5).

Если по видео или коду останутся вопросы, я могу пригласить вас на короткую онлайн-защиту на 5–10 минут: покажете приложение и ответите на пару вопросов по своему коду.

8. Важно

- Репозиторий обязательно должен быть публичным (Public), иначе я не смогу открыть ваш код.
- Pull Request по project должен быть открыт к моменту проверки, не мерджите его сами.
- Не удаляйте и не переименовывайте ветку main.
- Код должен быть вашим: в видео вы объясняете его своими словами. Пользоваться документацией, примерами с занятий и подсказками нейросетей можно, но вы должны понимать каждую часть своего проекта.
- Если на любом шаге что-то не работает, не удаляйте и не пересоздавайте всё с нуля, пришлите мне текст ошибки, разберёмся.